
CHARACTERIZING THE IMPACT OF PADDING TOKENS ON LLM INFERENCE EFFICIENCY UNDER DIFFERENT DIMENSIONS OF PARALLELISM

Chirag Adwani^{1*} Alex Coppens^{2*} Jinson Antony Jimmy^{2*}

ABSTRACT

During static batching, padding tokens are an unavoidable artifact of batched LLM inference, as batched sequences must be aligned to equal length to create rectangular tensors. This introduces padding tokens with no semantic content. The interaction between padding and different distributed parallelism strategies remains poorly characterized. In this report, we study how padding ratio and intra-batch length variance (measured by coefficient of variation, CoV) affect inference efficiency among three parallel configurations of Qwen2.5-32B on Megatron-LM using 4×A100-40GB GPUs. We sweep padding proportion and padding CoV using prompts drawn from ShareGPT, perform inference, and measure time to first token (TTFT), prefill and decode throughput (raw and effective), and peak GPU memory. We show that due to Megatron-LM’s prefill loop, prefill time scales with padding imbalance, causing TTFT to vary by up to 6× across batches with identical tensor shape. We determine the optimal parallel strategy for each configuration of batching statistics.

1 BACKGROUND AND MOTIVATION

Large Language Models (LLMs) have become increasingly central to a wide range of applications, from chatbots, to code writing assistants and scientific reasoning. Optimizing the efficiency of these models at inference time is a critical systems challenge, as any deployment of these models must satisfy latency and throughput requirements while managing the hardware costs.

Modern LLMs such as Qwen2.5-32B require upwards of 60 GB of memory just to store the model weights in `fp16`. When factoring in memory usage due to KV cache, this can exceed the capacity of even the largest commercially available GPU HBMs. This necessitates the usage of parallelism strategies such as Tensor Parallelism (TP) and Pipeline Parallelism (PP) at inference time, allowing inference to be performed across multiple devices in parallel. Under TP, weight matrices are sharded across multiple GPUs, requiring an all-reduce after every linear layer. Under Pipeline Parallelism, model layers are partitioned into sequential stages across GPUs. Activations are communicated via point-to-point transfers at stage boundaries. Unlike Data Parallelism (DP), which is embarrassingly parallel at inference time

(AWS Neuron Team, 2024), both TP and PP have collective communication as their critical steps, making them sensitive to workload characteristics.

A fundamental challenge in LLM inference is that real-world requests may arrive with highly variable prompt sequence lengths. During static batching, to form a batch with rectangular tensors for efficient GPU computation, shorter sequences in the batch are padded to the same length as the longest sequence in the batch. While this is simple to implement, the padding consumes computing power and memory while carrying no semantic importance (Jiang et al., 2024). In the prefill phase, padded tokens participate in the computations taking place in the attention block, the cost of which is $\mathcal{O}(L^2)$ in L , the sequence length (Vaswani et al., 2017). Furthermore, in the decode phase, padding also wastes KV cache slots increasing the memory pressure on an already constrained resource (inference is memory-bound) (Kwon et al., 2023).

The inefficiency introduced by padding is well understood in isolation and techniques such as iterative scheduling (as in ORCA (Yu et al., 2022)), and cellular batching (Gao et al., 2018) have been proposed to mitigate this issue. However, the interaction between padding overheads and distributed parallelism strategies at inference time for static batching remains uncharacterized, to the best of the knowledge of the authors.

This project aims to empirically characterize how different padding distributions (i.e. the % and variance of the number of pad tokens inside a batch) affect inference efficiency,

^{*}Equal contribution ¹Department of Mathematics, University of Maryland, College Park, MD, USA ²Department of Computer Sciences, University of Maryland, College Park, MD, USA. Correspondence to: Chirag Adwani <cadwani@umd.edu>, Alex Coppens <acoppens@umd.edu>, Jinson Antony Jimmy <jjimmy@umd.edu>.

measured in throughput (tokens/sec) and overall wall-clock time to produce a certain number of output tokens, across different TP and PP configurations. Our goal is to produce an empirical characterization that informs the people who deploy these models at a large scale, about the cost of padding across the different parallelism dimensions, and to motivate optimal parallelism configurations in production inference systems.

With this in mind, we can state the problem that we aim to study, in the form of an equation,

$$\Delta \text{Perf} = f(\text{pad_dist}, \text{TP_degree}, \text{PP_degree}),$$

where Perf is some performance metric, hence ΔPerf is the change in that metric, studied as a function f of pad_dist , the distribution of padding tokens inside a batch (i.e. % of pad tokens and their variance), TP_degree , the number of GPUs that we shard our tensors across (degree of TP parallelism), and PP_degree the number of GPUs that we shard our stages across (degree of PP parallelism).

2 METHODOLOGY

2.1 Data Generation and Pre-Processing Batches

To study the variable impact of padding on Tensor and Pipeline Parallelism strategies, we perform experiments in a sweep across batches with different $\text{pad\%}$ and pad_var . We source prompts from an open dataset and apply a statistically grounded sampling process to produce batches with the desired padding properties. We apply two different batch construction strategies to produce two runs: v1 which stabilizes real token count per batch constant and v2 which holds total token count per batch constant.

2.1.1 Dataset

We use the ShareGPT Vicuna Unfiltered dataset (Chiang et al., 2023), a large collection of real user-chatbot conversations scraped from the ShareGPT platform. From each conversation, only the first human turn is extracted as the inference prompt, since we only benchmark single-turn prefill performance. This yields 92,792 candidate prompts. We tokenize all the prompts using the Qwen2.5-32B tokenizer with `add_special_tokens = False`, so that token counts reflect the prompt body only. This is consistent with how Megatron-LM prepends special tokens at inference time.

After tokenization, for v1 we filter to prompts with length in $[32, 1024]$, retaining 34,757 prompts, and for v2 we filter to prompts with length in $[1, 128]$ tokens, retaining 75,019 prompts.

2.1.2 Controlling Padding Ratio and Intra-Batch variance

We want to vary two dimensions of padding distribution within each batch. The first is the padding ratio ($\text{pad\%}$) which quantifies what percentage of the tokens within a batch are pad tokens. Equivalently, this is the mean percentage of tokens which are padding over the sequences of a batch. The second is the Coefficient of Variation (CoV) of sequence padding percentage, which quantifies how uneven the sequence lengths within the batch are. We construct a 4×4 grid of experimental configurations spanning four $\text{pad\%}$ levels: 20%, 30%, 50%, 70%, and four CoV levels: 0.05, 0.20, 0.35, 0.5 for v1 and 0.15, 0.25, 0.35, 0.45 for v2, for a total of 16 conditions for each of v1 and v2. For each condition, we construct five batches of 128 prompts each.

For formalism, we define $\text{pad\%}$ as the fraction of token slots in a batch occupied by [PAD] tokens. This can be calculated as

$$\text{pad\%} = \frac{B \cdot L_{\max} - \sum_{i=1}^B \ell_i}{B \cdot L_{\max}},$$

where B is the batch size, ℓ_i is the token length of the i^{th} sequence.

We define CoV as the coefficient of variation of padding proportion within a batch. This can be calculated as

$$\text{CoV} = \frac{\sigma}{\text{pad\%}}.$$

where σ is the standard deviation of padding proportion within the batch calculated as

$$\sigma = \sqrt{\frac{1}{B} \sum_{i=1}^B \left(\frac{L_{\max} - \ell_i}{L_{\max}} - \text{pad\%} \right)^2}$$

CoV captures how uniformly or unevenly padding is distributed across sequences in the batch. Two batches with identical $\text{pad\%}$ can differ substantially in CoV: sequences in a low-CoV batch will have very similar padding percentage and therefore sequence length, while a high CoV batch will have a large range of sequence lengths.

2.1.3 Batch Construction algorithm

For v1, to stabilize the real token count in all batches, we select the target mean length for all batches, L_{avg} , and select an anchor sequence to be the longest sequence in the batch with length $L_{\max} = \frac{L_{\text{avg}}}{\text{pad\%}}$. We then select $B - 1$ batch elements by sampling $B - 1$ lengths from a normal distribution $\mathcal{N}(\mu = L_{\text{avg}}, \sigma^2 = \text{CoV} \cdot L_{\text{avg}})$. Sampled lengths greater than $L_{\max}$ are discarded. We iteratively select the prompt with the closest length to the sampled lengths without replacement to form the batch.

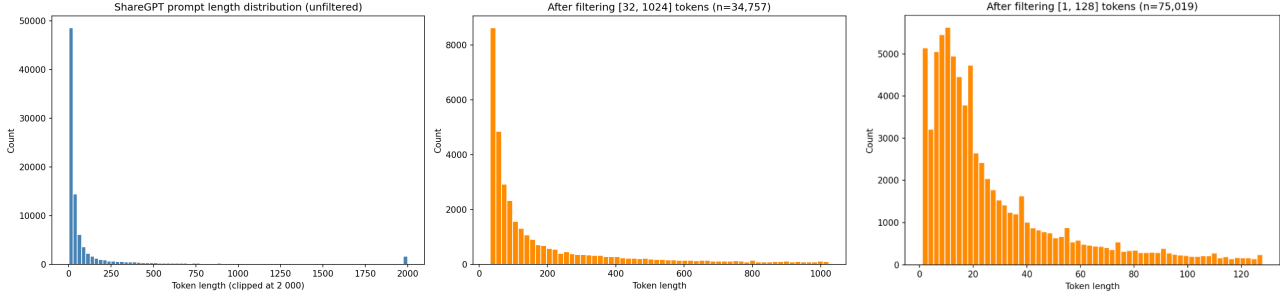


Figure 1. Token length distribution of ShareGPT prompts before and after filtering. (Left) Distribution of all 92,792 extracted prompts tokenized with Qwen2.5-32B tokenizer, clipped at 2000 tokens for readability. The small bar at the end is for the tokens that have ≥ 2000 tokens. Otherwise the maximum length of sequences go up to 221,043 tokens. The distribution is strongly right-skewed, with a median of 24 tokens. (Middle) The 75,019 prompts retained after filtering to the $[32, 1024]$ token range. The skew still persists, with majority of prompts concentrated below 100 tokens. (Right) The 75,019 prompts retained after filtering to the $[1, 128]$ token range.

For v2, to equalize the total token count in all batches, we fix the size of the anchor sequence to $L_{\max} = 128$ tokens and sample a sequence with this length from the dataset. We can model the selection of the remaining $B - 1$ sequences in the batch as sampling from a Beta distribution. The beta distribution is a continuous distribution with support on the interval $[0, 1]$ that is parametrized by two parameters α and β . We draw $B - 1$ samples from the Beta distribution to select the padding proportions and thus the sequence length of the $B - 1$ remaining sequences.

For a given batch with target $\text{pad\%} = \mu$ and target $\text{CoV} = \frac{\sigma}{\mu}$ we have target variance σ^2 . The anchor element has padding proportion 0 which has the effect of biasing the sample mean downwards and the sample variance upwards (because 0 is an extremal value in the Beta distribution). To compensate, we produce an adjusted target mean and variance for the $B - 1$ remaining samples. We produce the adjusted target mean as follows

$$\mu = \frac{1}{B} \times 0 + \frac{B-1}{B} \times \mu' \implies \mu' = \frac{B}{B-1} \mu$$

and we produce the target variance as follows

$$\begin{aligned} \sigma^2 &= \frac{(0 - \mu')^2}{B} + \frac{B-1}{B} (\sigma')^2 = \frac{(\mu')^2}{B} + \frac{B-1}{B} (\sigma')^2 \\ \implies (\sigma')^2 &= \frac{B\sigma^2 - (\mu')^2}{B-1} \end{aligned}$$

This allows us to calculate the parameters α and β for the Beta distribution using the following identities:

$$\nu = \frac{\mu'(1 - \mu')}{(\sigma')^2} - 1; \quad \alpha = \mu' \times \nu; \quad \beta = (1 - \mu') \times \nu$$

After sampling $B - 1$ elements p_i from the distribution $\text{Beta}(\alpha, \beta)$, we re-normalize to sequence lengths by setting

the lengths to $L_i = p_i \times L_{\max}$ and for each L_i we select a sequence from the dataset with length L_i without replacement. As a natural result of forming a sample, there is a small bias in the actual statistics of the sample padding versus the targeted statistics. We quantify this as error in figure 2.

2.2 Inference Procedure

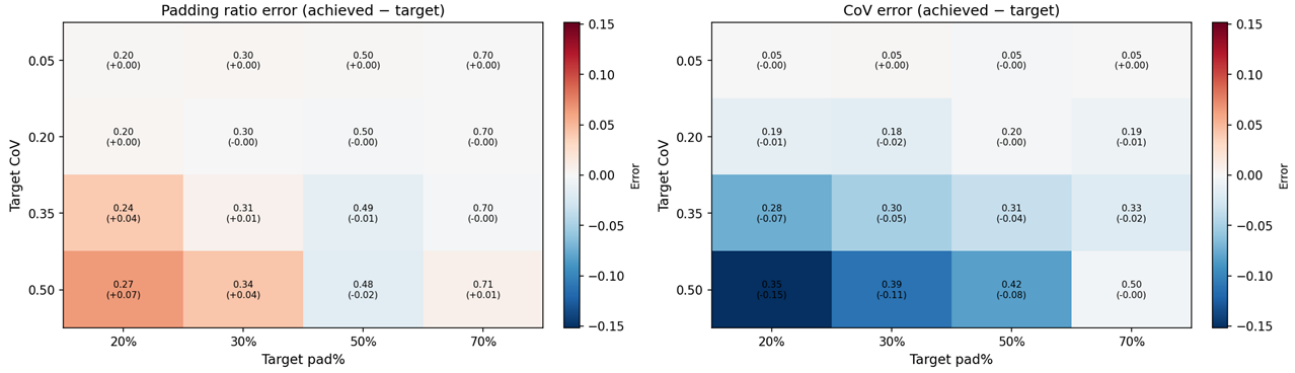
We perform inference using the Megatron inference engine. We sweep over a 4×4 grid of experimental conditions for two batching methods, varying $\text{pad\%} \in \{20\%, 30\%, 50\%, 70\%\}$, and $\text{CoV} \in \{0.05, 0.20, 0.35, 0.5\}$ for v1 and $\text{CoV} \in \{0.15, 0.25, 0.35, 0.45\}$ for v2. For each condition, we load a dataset of pre-processed batches with the desired padding properties. We run two warmup batches before running timed batches over the rest of the dataset. Decoding is done greedily ($\text{top.k} = 1$). For each batch, we record the TTFT, prefill throughput, decode throughput, per-real-token latency, and peak memory usage.

Our inference benchmark is duplicated over three configurations: 4-way tensor parallelism and no pipeline parallelism (`config_a`), 2-way pipeline parallelism and 2 way tensor parallelism (`config_b`), and 4-way pipeline parallelism and no tensor parallelism (`config_c`). The configuration parameters are provided in the startup script and are consumed by the MegatronLM static inference engine.

Each timed batch is measured with two separate `generate()` calls to isolate prefill from decode latency. Prior to each call, all CUDA kernels are synchronized to ensure the wall-clock measurement excludes queued GPU work. The first call generates a single output token and its elapsed time is recorded as the *time to first token* (TTFT), capturing total prefill cost plus one decode step; this is the primary locus of padding overhead. The second call gen-

v1

Batching accuracy across the 4x4 benchmark grid
 Blue = undershot target, Red = overshoot target, White = exact

**v2**

Batching accuracy across the 4x4 benchmark grid
 Blue = undershot target, Red = overshoot target, White = exact

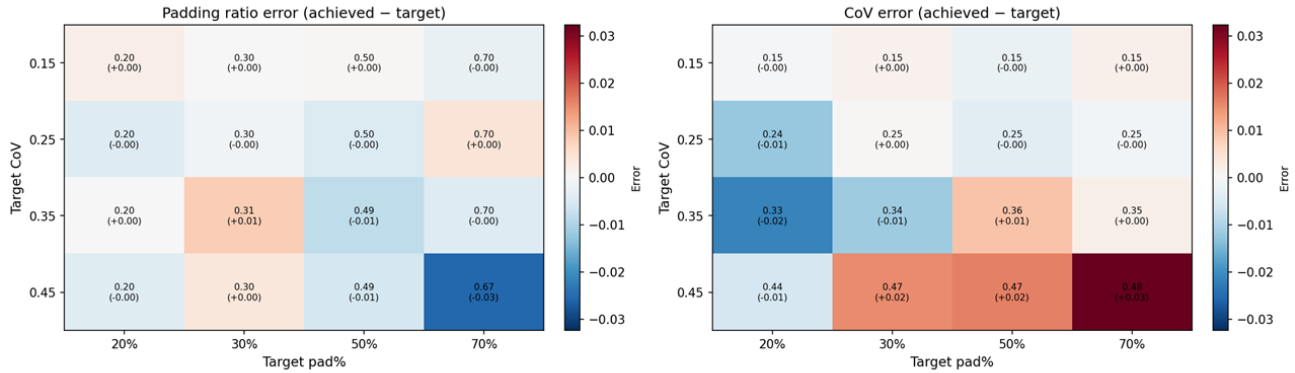
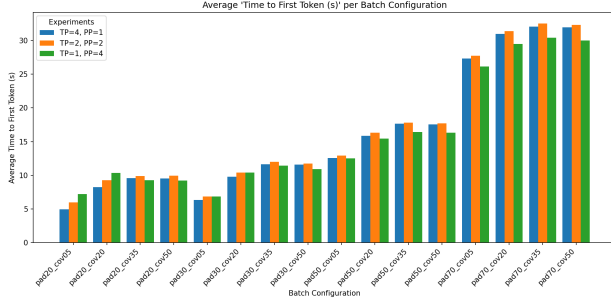
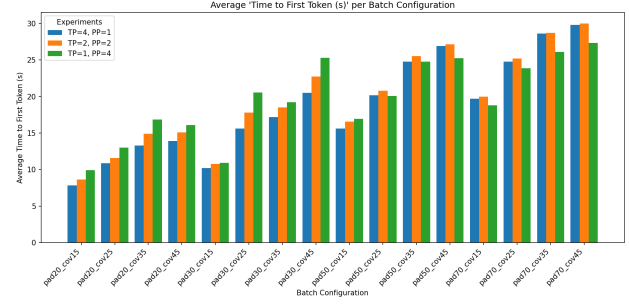


Figure 2. Batching accuracy across the 4×4 experimental grid. Each cell reports the mean achieved value across the five batches constructed for that condition, along with the signed error relative to the target (in parentheses). Color encodes error on a diverging scale, with red corresponding to positive error(overshoot), white indicating no error, and blue indicating negative error(undershoot). (*Top-Left*) Error in padding ratio for v1 batches: The pad% targets are all met within ± 0.01 for 30%, 50%, and 70% conditions across all CoV levels. A mild overshoot of 0.07 is observed at high CoV, due to the nature of the distributions. (*Top-Right*) Error in CoV for v1 batches: the two lower CoV targets are well controlled across all pad levels, higher CoV levels systematically undershoot at low pad levels because there simply aren't that many long sequences in the right tail of the ShareGPT distribution. (*Bottom-Left*) Error in padding ratio for v2 batches. (*Bottom-Right*) Error in CoV for v2 batches. We note that the error range for v2 batches is substantially smaller than that for v1 batches. Overall, the errors are tolerable and the overall achieved metrics of the data should represent a comprehensive sweep over the two variables. We shall report our results with respect to these obtained values.

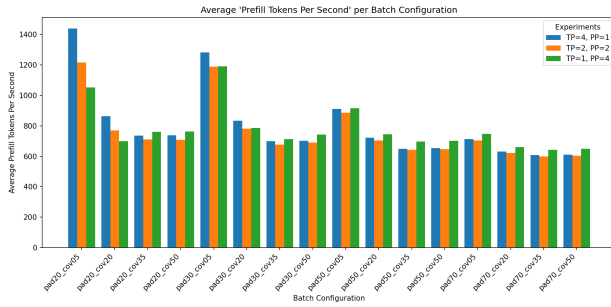


(a) **v1** (similar real token count across cells, total tokens in a cell grow from left to right).

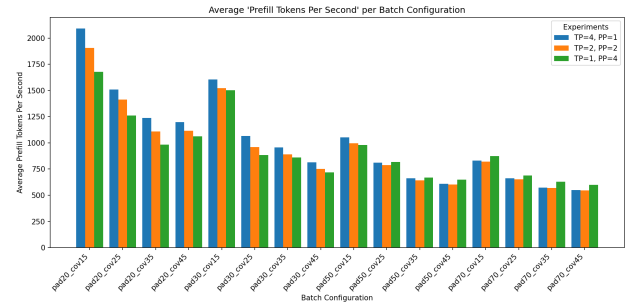


(b) **v2** (same number of total tokens across cells, number of real tokens shrink from left to right).

Figure 3. Mean TTFT (prefill + 1 decode step) per benchmark cell, grouped by parallelism configuration. The x -axis labels encode each cell as `pad<P>_cov<C>`; lower is better. In v2, despite a constant padded tensor shape across all cells, `config_a` TTFT spans 7.8-29.8 s.



(a) **v1** (similar real token count across cells, total tokens in a cell grow from left to right).



(b) **v2** (same number of total tokens across cells, number of real tokens shrink from left to right).

Figure 4. Mean prefill throughput ($\text{total_tokens}/\text{ttft_s}$) per cell; higher is better. Prefill TPS is the direct dual of TTFT and inherits the same spread-driven collapse: v2 `config_a` drops from ~ 2090 tok/s at `pad20_cov15` to ~ 550 tok/s at `pad70_cov45`.

erates $n_{\text{decode}} = 512$ output tokens and its elapsed time t_{decode} is recorded as the *decode wall-clock time*.

We use two throughput measures. *Prefill raw throughput* defined as

$$\text{TPS}_{\text{prefill}} = \frac{B \cdot L_{\text{max}}}{t_{\text{TTFT}}}$$

and *decode throughput* defined as

$$\text{TPS}_{\text{decode}} = \frac{B \cdot n_{\text{decode}}}{t_{\text{decode}}}$$

To control for the fact that high-pad batches contain fewer “useful” prompt tokens, we report TTFT divided by the number of real tokens (*time per real input token*):

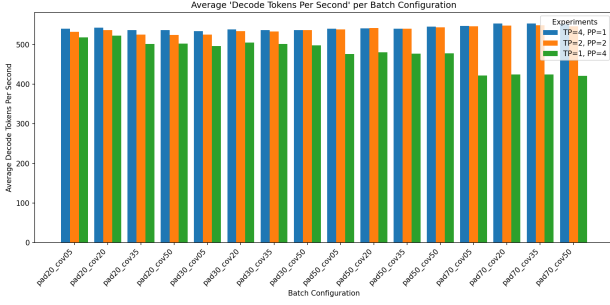
$$\text{TPRIT} = \frac{1000 \times t_{\text{TTFT}}}{\sum_i L_i}$$

where the units for TPRIT are ms/real-token. t_{TTFT} is recorded in seconds.

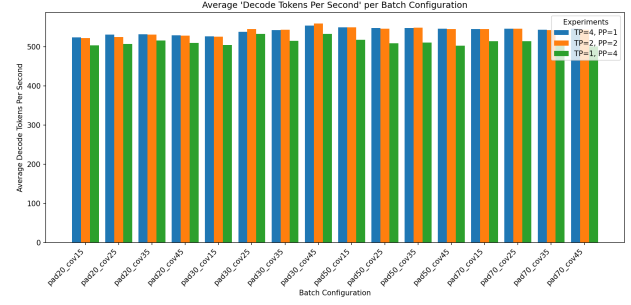
Peak GPU memory is recorded per batch and reset between measurements. Even though this metric was recorded, we did not observe any interesting trends across batches. There was a trend across configs, which can be attributed to minor differences in shard sizes across the devices.

3 RESULTS

We benchmark Qwen2.5-32B on 4 x A100-40GB across three parallelism configurations: `config_a` (TP=4, PP=1), `config_b` (TP=2, PP=2), and `config_c` (TP=1, PP=4) using Megatron-LM’s `StaticInferenceEngine`. Two sweep designs were run: **v1** (16 cells $\times$ 5 batches $\times$ 64 prompts, $\text{CoV} \in \{0.05, 0.20, 0.35, 0.50\}$, prompt lengths between 32 and 305 tokens) and **v2** (16 cells $\times$ 5 batches $\times$ 128 prompts, $\text{CoV} \in \{0.15, 0.25, 0.35, 0.45\}$, prompts truncated to a max of 128 tokens). The pad-ratio axis is identical across versions: $\{0.20, 0.30, 0.50, 0.70\}$.

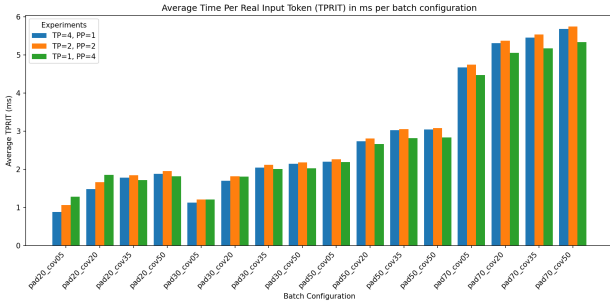


(a) **v1** (similar real token count across cells, total tokens in a cell grow from left to right).

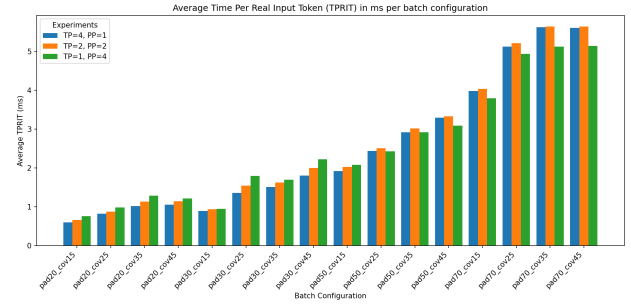


(b) **v2** (same number of total tokens across cells, number of real tokens shrink from left to right).

Figure 5. Mean decode throughput (generated_tokens/decode_time) per cell; higher is better. Decode TPS is essentially flat across cells because each decode step operates on a fixed $[B, 1]$ shape, independent of the original prompt-length distribution.



(a) **v1** (similar real token count across cells, total tokens in a cell grow from left to right).



(b) **v2** (same number of total tokens across cells, number of real tokens shrink from left to right).

Figure 6. Time Per Real Input Token (TPRIT), this is TTFT divided by the number of real (non-pad) prompt tokens in milliseconds per real token ($1000(\text{ms}/\text{s}) \times \text{ttft_s}/\text{real_tokens}$); lower is better. This metric compounds rising TTFT with shrinking `real_tokens` and spans roughly an order of magnitude (~ 0.6 to ~ 6 ms/tok) across the v2 grid.

3.1 Time-to-first-token

We present TTFT as the total prefill time followed by one decode step. For the v1 runs, obtained mean TTFT is 15.8 – 16.6s across the three configs; for v2 the mean TTFT is 18.7 – 19.7s. Figure 3 shows the TTFT per configuration.

The dominant feature in both sweeps is that the TTFT rises monotonically with both `pad%` and `CoV`. In v2, TTFT goes from 7.8s at the cheapest cell (pad20_cov15) to 29.8s at the most heterogeneous (pad70_cov45) under `config_a`, which is a $3.8\times$ gap on what is, by construction, a constant shape $(128, 128, d_{emb})$ padded tensor ($B \times \max(\text{prompt_len}) = 16, 384$ for v2). In a standard setting, Attention layer FLOPs scale as $\mathcal{O}(S_{pad}^2)$, where S_{pad} is the padding size. But Megatron-LM’s prefill is done in two phases:

- **Bulk-Prefill Phase:** Prefill all B batches in parallel, for positions between 0 and `min_prompt_length_in_batch`.
- **Iterative-Prefill Phase:** A loop from minimum prompt

length to max prompt length inside a batch, advancing one position per step. Each step is $[B, 1]$ decode-style forwarding through all model layers, paying full per-layer collective overhead with low arithmetic intensity.

So, the limiting step is the *iterative-prefill* stage which scales with the prompt-length spread which is $L_{\max} - L_{\min}$. To characterize the impact that this prompt-length spread has on TTFT, we fit the following linear model:

$$TTFT \approx \text{intercept} + \text{slope} \times \text{spread}$$

to each configuration on the v2 dataset.

config_	Slope (s/step)	Intercept (s)	R^2
a (TP=4,PP=1)	0.199	1.50	0.92
b (TP=2,PP=2)	0.189	3.24	0.89
c (TP=1,PP=4)	0.147	6.99	0.76

Both the slope and the intercept have a physical interpretation. The *slope* is the latency of a single iterative-fill $[B, 1]$



Figure 7. Per-cell winning parallelism configuration for TTFT (lower is better). Cell color denotes the configuration with the lowest mean TTFT; the numeric annotation shows the winning value in seconds. The ranking flip between v1 and v2 reflects the shift from bulk-prefill-dominated to iterative-prefill-dominated cost.

step, while the *intercept* reflects the bulk-prefill cost. We can note that both *pad%* and *CoV* axes increase the spread (*pad%* lowers $L_{\min}$, while $L_{\max}$ is held at 128), which is why we see that both axes increase the TTFT numbers.

We can also note from this regression analysis that PP=4 pays the least per step in iterative-prefill overhead, while TP=4 pays the most because of its higher number collective communications (PP=4 slope is higher, while TP=4 slope is lower). Whereas, PP=4 pays the most during bulk-prefill stage due to pipeline bubbles. A single bulk chunk cannot be amortised across micro-batches. Whereas TP=4 has no bubbles, so it wins out during bulk-prefill.

3.2 Prefill and Decode throughput

Prefill raw TPS and decode TPS are shown in Figs 4, 5. Prefill TPS displays the same trend as TTFT but inverted, as the two metrics are directly inversely related; v2 prefill_raw_tps drops from ~ 2090 tok/s at pad20_cov15 to ~ 550 tok/s at pad70_cov45, a $4\times$ degradation.

Decode TPS is comparatively stable across configurations. Run v1 averages 542/538/478 tok/s for config_a / config_b / config_c; run v2 averages: 540/540/512 tok/s. This is because the decode loop operates on a $[B, 1]$ slice regardless of prompt heterogeneity. The workload shape during each decode is independent of the original padding pattern.

3.3 Time per real input token

Time per real input token (Figure 6) is compounded by two effects: rising TTFT and shrinking *real_tokens*, and summarizes of “how much time is spent in prefill for each non-padding token” In v2, it ranges from ~ 0.6 ms/tok at low *pad/cov* to ~ 6 ms/tok at high *pad/cov*, an order of magnitude swing on a workload that the system reports as having identical padded shape.

3.4 Best configurations

Figs 7, and 8 show, per configuration, which parallelism configuration has the lowest TTFT and highest decode TPS, respectively. For TTFT, the optimal parallel strategy varies per run. For run v1, config_a (TT=4) has the lowest TTFT for configurations with low *pad%* and low *CoV* and config_c (PP=4) has the lowest TTFT for all other configurations. For run v2, config_a (TT=4) has the lowest TTFT for configurations with low *pad%* and config_c (PP=4) has the lowest TTFT for high *pad%*. config_b (TP=2, PP=2) has suboptimal TTFT for both runs and for all configurations. The change in optimal parallel strategy between v1 and v2 is informative. With uncapped prompts, the bulk-prefill phase processes larger (B , $\max_len$, d_{emb}) tensors, making bulk-prefill the dominant contributor to TTFT. In this regime, PP=4 wins because its activation-passing traffic across layers is simply much smaller than TP=4’s per-layer all-reduce volume. The large prefill chunks of bulk-prefill amortises PP=4’s pipeline bubble across many tokens and its collective communication overhead is lower.

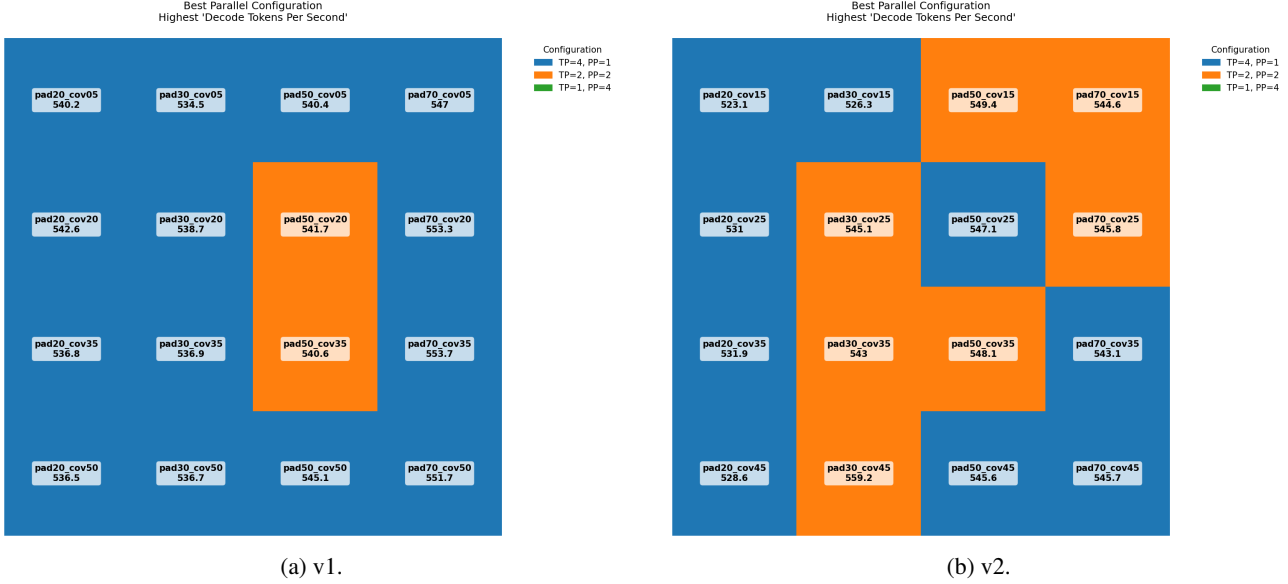


Figure 8. Per-cell winning configuration for decode throughput (higher is better).

However, as noted earlier, with prompts capped at 128 and batch size doubled to 128 in v2, the bulk-prefill compute drops and iterative-prefill latency takes over. `config_a` (TP=4) has lower TTFT for more configurations here because the per-step $(B, 1, d_{emb})$ collectives are smaller while `config_c` (PP=4) suffers from pipeline bubble. `config_b` never wins TTFT in either sweep as it is consistently dominated by one of the pure-parallelism strategies.

For decode TPS, `config_a` (TP=4) wins 14 out of 16 cells in v1 and 9 out of 16 in v2. Interestingly, `config_b` takes 2 and 7, respectively while `config_c` (PP=4) never wins decode tps in either sweep. This is consistent with the fact that every decode step is $[B, 1]$ forward, as pointed out earlier, identical to iterative-prefill step but without any bulk chunk to amortise against. PP=4’s per-step pipeline bubble therefore dominates, and this time there is no compute to hide that cost, while TP=4’s per-step collectives stay tiny. The cross-configuration spread is also much narrower for decode (at most $\sim 24\%$ in v1, $\sim 8\%$ in v2) than for TTFT (which spans $4\times$ across v2 grid, reflecting decode’s structural insensitivity to prompt-length heterogeneity. Every step has the same size, regardless of pad% or CoV.

4 CONCLUSIONS AND FUTURE WORK

We have characterized the inference performance for Qwen2.5-32B across a sweep padding proportion and CoV under three parallel configurations, and identified that the cost of variable prompt lengths for Megatron-LM is dominated by the iterative-prefill loop scaling with prompt length spread. The optimal TP/PP choice depends on whether bulk-prefill or iterative-prefill dominates, and this in turn depends

on the dataset shape. This highlights the importance of being aware of the workload being processed.

Several extensions naturally flow from this work. Cross-model and cross-architecture generalization is one obvious direction. Llama-3-70B, Gemma-2-27B, and MoE models (Mixtral, DeepSeek-V3) all have different architectural hyperparameters; MoE in particular could magnify the iterative-fill cost because each $(B, 1)$ step now also pays an all-to-all dispatch/combine. Whether our characterizations are model-specific or mechanism-specific is currently unknown. Another question is whether the same characterization transfers to training, where padding interacts with gradient accumulation rather than an iterative-fill loop and optimizer-state memory replaces activation memory as the binding constraint.

One could also build an empirical cost model parametrized over $(B, L_{\max}, L_{\min}, TP, PP)$ that predicts TTFT for unseen configurations, letting us reason about larger clusters (more than one node) without performing a full-sweep. This is particularly valuable for larger clusters where collective performance is bottle-necked by inter-node bandwidth.

Finally, the metrics reported here are batch-aggregate and treat the engine as a black box. Profiling bulk-prefill and iterative-fill separately would strengthen the characterization.

5 CONTRIBUTIONS

Alex Coppens produced code for batch generation for the v2 experiment, and for experimental data visualization. He also performed debugging runs for inference experiment

scripts using Qwen2.5-0.5B on one A100 multi-instance GPU (one seventh of an A100).

Chirag Adwani generated the script and batches for the v1 experiment, and set up the instance of Megatron-LM on Zaratan. He was responsible for pushing the jobs to be run with the Qwen2.5-32B model on 4 A100 GPUs on Zaratan, and then extracting the results.

Jinson Antony Jimmy worked on the Megatron-based padding benchmark script as well as the supporting launch scripts for multiple parallelism configs, and a full results aggregation/plotting script to generate summaries and heatmaps. He also attempted running and debugging the scripts on the Zaratan cluster.

AI USAGE DISCLOSURE

AI was used to produce portions of the batch generation, inference, regression, and data visualization scripts.

ACKNOWLEDGMENTS

Thank you to Lannie Dalton Hough for discussing the details of our project and helping us solidifying our ideas.

GITHUB REPOSITORY

The public Git Repo for the project can be found here: <https://github.com/divide-by-zero/llm-inference-padding-study>

REFERENCES

- AWS Neuron Team. Parallelism techniques for LLM inference. AWS Neuron Documentation, 2024. URL <https://awsdocs-neuron.readthedocs-hosted.com/en/latest/libraries/nxd-inference/app-notes/parallelism.html>.
- Chiang, W.-L., Li, Z., Lin, Z., Sheng, Y., Wu, Z., Zhang, H., Zheng, L., Zhuang, S., Zhuang, Y., Gonzalez, J. E., Stoica, I., and Xing, E. P. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL <https://lmsys.org/blog/2023-03-30-vicuna/>.
- Gao, P., Yu, L., Wu, Y., and Li, J. Low latency rnn inference with cellular batching. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450355841. doi: 10.1145/3190508.3190541. URL <https://doi.org/10.1145/3190508.3190541>.
- Jiang, C., Jia, Z., Zheng, S., Wang, Y., and Wu, C. DynaPipe: Optimizing multi-task training through dynamic pipelines. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, pp. 1–16, 2024. doi: 10.1145/3627703.3629585.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-28-1. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.